

Lesson 6 Notes: Lower Triangular Matrix Column-Major Mapping

1. Main idea of Lesson 6

Lesson 6 teaches **lower triangular matrix column-major mapping**.

In Lesson 5, we learned how to store a lower triangular matrix using **row-major order**.

In Lesson 6, the matrix type is still the same:

Lower triangular matrix

The stored positions are still the same:

$i \geq j$

The zero positions are still the same:

$i < j$

The main new idea is the **storage order**.

In Lesson 5, we stored values **row by row**.

In Lesson 6, we store values **column by column**.

So Lesson 6 is not about a new matrix shape. It is about a new way to arrange the stored lower triangular values inside a 1D array.

2. What you should already know before Lesson 6

Before Lesson 6, you should understand these ideas:

Concept	Meaning
Special matrix	A matrix with a useful pattern that lets us avoid storing unnecessary values.
Structural zero	A zero that exists because of the matrix rule, not because we stored it.

Concept	Meaning
1D array storage	Instead of storing the full matrix, we store only useful values in one array.
Lower triangular matrix	A matrix that stores values on and below the main diagonal.
Row-major storage	Store values row by row.
Column-major storage	Store values column by column.
Matrix indexing	Matrix positions use row <code>i</code> and column <code>j</code> , usually starting at 1.
Array indexing	C and C++ arrays start at index 0.

The most important Lesson 5 formula was the lower triangular row-major formula:

$$\text{index} = i(i - 1)/2 + (j - 1)$$

That formula works for **row-major** storage.

Lesson 6 uses a different formula because the storage order changes to **column-major**.

3. Quick reminder: what is a lower triangular matrix?

A lower triangular matrix stores meaningful values on and below the main diagonal.

Example:

```

1  0  0  0  0
2  3  0  0  0
4  5  6  0  0
7  8  9  10 0
11 12 13 14 15

```

The useful part is the lower-left triangle:

```

1
2  3
4  5  6
7  8  9  10
11 12 13 14 15

```

The values above the main diagonal are zeros.

4. Stored condition

For a lower triangular matrix, a position is stored when:

```
i >= j
```

Where:

```
i = row number  
j = column number
```

Examples:

```
M[1,1] is stored because 1 >= 1  
M[2,1] is stored because 2 >= 1  
M[2,2] is stored because 2 >= 2  
M[4,2] is stored because 4 >= 2  
M[5,3] is stored because 5 >= 3
```

These positions are on or below the main diagonal.

5. Zero condition

For a lower triangular matrix, a position is a structural zero when:

```
i < j
```

Examples:

```
M[1,2] is zero because 1 < 2  
M[1,5] is zero because 1 < 5  
M[2,4] is zero because 2 < 4  
M[3,5] is zero because 3 < 5
```

These positions are above the main diagonal.

They are not stored in the array.

The program simply returns or prints `0` when needed.

6. Lower triangular storage size

A normal $N \times N$ matrix stores:

N^2 values

A lower triangular matrix stores only:

$N(N + 1) / 2$ values

This is because row or column storage includes exactly the lower triangle.

For $N = 5$:

Stored values = $5(5 + 1) / 2$
Stored values = $5(6) / 2$
Stored values = $30 / 2$
Stored values = 15

So a 5×5 lower triangular matrix stores 15 values instead of 25.

7. Structural zero count

A full $N \times N$ matrix has:

N^2 total positions

A lower triangular matrix stores:

$N(N + 1) / 2$ positions

So the number of structural zeros above the diagonal is:

$N(N - 1) / 2$

For $N = 5$:

```
Zero positions = 5(5 - 1) / 2
Zero positions = 5(4) / 2
Zero positions = 20 / 2
Zero positions = 10
```

So a **5 x 5** lower triangular matrix has:

```
15 stored values
10 structural zeros
25 total positions
```

8. Row-major vs column-major

Lesson 5 used row-major storage.

Lesson 6 uses column-major storage.

The matrix is the same, but the array order is different.

9. Row-major storage reminder

Row-major means:

```
Store row 1
Then row 2
Then row 3
Then row 4
...
```

Example matrix:

```
1  0  0  0
2  3  0  0
4  5  6  0
7  8  9  10
```

Row-major storage:

```
Row 1: 1
Row 2: 2, 3
Row 3: 4, 5, 6
Row 4: 7, 8, 9, 10
```

So the array becomes:

```
A = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

The row-major formula is:

```
index = i(i - 1)/2 + (j - 1)
```

This formula skips previous rows.

10. Column-major storage

Column-major means:

```
Store column 1
Then column 2
Then column 3
Then column 4
...
```

For a lower triangular matrix, each column starts at the diagonal and goes downward.

Example `5 x 5` lower triangular matrix:

```
1  0  0  0  0
2  3  0  0  0
4  5  6  0  0
7  8  9  10 0
11 12 13 14 15
```

Column-major storage means:

```
Column 1: 1, 2, 4, 7, 11
Column 2: 3, 5, 8, 12
```

```
Column 3: 6, 9, 13
Column 4: 10, 14
Column 5: 15
```

So the 1D array becomes:

```
A = [1, 2, 4, 7, 11, 3, 5, 8, 12, 6, 9, 13, 10, 14, 15]
```

This array stores the same matrix as row-major storage, but in a different order.

11. Column sizes in lower triangular column-major storage

For an $N \times N$ lower triangular matrix:

```
Column 1 stores N values
Column 2 stores N - 1 values
Column 3 stores N - 2 values
Column 4 stores N - 3 values
...
Column N stores 1 value
```

For $N = 5$:

```
Column 1 stores 5 values
Column 2 stores 4 values
Column 3 stores 3 values
Column 4 stores 2 values
Column 5 stores 1 value
```

This happens because lower triangular values start at the diagonal and continue downward.

Column 1 starts at row 1, so it has many values.

Column 5 starts at row 5, so it has only one value.

12. Main mental model for Lesson 6

To find the array index of $M[i, j]$ in column-major storage:

Step 1: Check if $i \geq j$.
Step 2: Skip all stored values in the columns before column j .
Step 3: Move downward inside column j until you reach row i .

In simple words:

$\text{index} = \text{skipped columns} + \text{movement inside current column}$

This is the easiest way to understand the formula.

13. Why we check $i \geq j$ first

Before calculating the index, always check:

$i \geq j$

If this is true, the position is stored.

If this is false, the position is above the diagonal and is not stored.

Example:

$M[2, 5]$

Here:

$i = 2$
 $j = 5$

Check:

$2 \geq 5$

This is false.

So $M[2, 5]$ is a structural zero.

The program should return 0 , or a helper index function may return -1 to mean "not stored."

14. Column-major index formula

For lower triangular column-major storage:

$$\text{index} = n(j - 1) - ((j - 2)(j - 1) / 2) + (i - j)$$

Use this formula only when:

$$i \geq j$$

If:

$$i < j$$

then the position is not stored.

15. Breaking the formula into two parts

The full formula is:

$$\text{index} = n(j - 1) - ((j - 2)(j - 1) / 2) + (i - j)$$

This can be written as:

$$\text{index} = \text{skipColumns} + \text{moveDown}$$

Where:

$$\begin{aligned} \text{skipColumns} &= n(j - 1) - ((j - 2)(j - 1) / 2) \\ \text{moveDown} &= i - j \end{aligned}$$

So the formula is not random. It has a clear meaning.

16. Part 1: skipColumns

The first part is:

$$\text{skipColumns} = n(j - 1) - ((j - 2)(j - 1) / 2)$$

This counts how many stored values come before column j .

For example, in a 5×5 lower triangular matrix:

Column 1 has 5 values
Column 2 has 4 values
Column 3 has 3 values
Column 4 has 2 values
Column 5 has 1 value

If the target is in column 4, we skip columns 1, 2, and 3:

$$5 + 4 + 3 = 12$$

So for column 4:

$$\text{skipColumns} = 12$$

17. Why the skipColumns formula works

Before column j , there are:

$$j - 1 \text{ columns}$$

If every previous column had n values, then we would have:

$$n(j - 1)$$

But the columns are not all full.

Column 1 has n values.

Column 2 has $n - 1$ values.

Column 3 has $n - 2$ values.

So we must subtract the missing upper-triangle part from the previous columns.

That missing count is:

$$((j - 2)(j - 1)) / 2$$

So:

$$\text{skipColumns} = n(j - 1) - ((j - 2)(j - 1) / 2)$$

This gives the number of stored values before column j .

18. Part 2: `moveDown`

The second part is:

$$\text{moveDown} = i - j$$

This tells us how far to move downward inside the current column.

In a lower triangular matrix, column j starts at row j .

Example: column 4 starts at:

$$M[4, 4]$$

Then the next stored position is:

$$M[5, 4]$$

So:

```
M[4,4] has moveDown = 4 - 4 = 0
M[5,4] has moveDown = 5 - 4 = 1
```

That is why:

```
moveDown = i - j
```

19. Formula in plain English

The formula:

```
index = n(j - 1) - ((j - 2)(j - 1) / 2) + (i - j)
```

means:

```
First, skip all stored values in the columns before column j.
Then, move downward inside column j until you reach row i.
```

That is the core idea of Lesson 6.

20. Visual mapping for a 5 x 5 matrix

Matrix:

```
1  0  0  0  0
2  3  0  0  0
4  5  6  0  0
7  8  9  10 0
11 12 13 14 15
```

Column-major array:

```
Index: 0  1  2  3  4  5  6  7  8  9  10 11 12 13 14
Value: 1  2  4  7 11 3  5  8 12 6  9 13 10 14 15
```

Mapping:

```
M[1,1] -> A[0]
M[2,1] -> A[1]
M[3,1] -> A[2]
M[4,1] -> A[3]
M[5,1] -> A[4]

M[2,2] -> A[5]
M[3,2] -> A[6]
M[4,2] -> A[7]
M[5,2] -> A[8]

M[3,3] -> A[9]
M[4,3] -> A[10]
M[5,3] -> A[11]

M[4,4] -> A[12]
M[5,4] -> A[13]

M[5,5] -> A[14]
```

21. Dry run 1: Find index of `M[4,4]` in a 5 x 5 matrix

We want:

```
M[4,4]
```

So:

```
n = 5
i = 4
j = 4
```

Step 1: Check condition

```
i >= j
4 >= 4
true
```

So the position is stored.

Step 2: Skip columns before column 4

Before column 4, we have columns 1, 2, and 3.

```
Column 1 has 5 values  
Column 2 has 4 values  
Column 3 has 3 values
```

So:

```
skipColumns = 5 + 4 + 3  
skipColumns = 12
```

Step 3: Move down inside column 4

Column 4 starts at:

```
M[4,4]
```

We are already at `M[4,4]`.

So:

```
moveDown = 0
```

Using the formula:

```
moveDown = i - j  
moveDown = 4 - 4  
moveDown = 0
```

Step 4: Add both parts

```
index = skipColumns + moveDown  
index = 12 + 0  
index = 12
```

So:

```
M[4,4] -> A[12]
```

22. Dry run 2: Find index of `M[5,4]`

We want:

```
M[5,4]
```

So:

```
n = 5  
i = 5  
j = 4
```

Step 1: Check condition

```
i >= j  
5 >= 4  
true
```

So the position is stored.

Step 2: Skip columns before column 4

Before column 4, we skip columns 1, 2, and 3.

```
Column 1 + Column 2 + Column 3  
= 5 + 4 + 3  
= 12
```

So:

```
skipColumns = 12
```

Step 3: Move down inside column 4

Column 4 starts at:

`M[4,4]`

The target is:

`M[5,4]`

So we move down one step.

Using the formula:

```
moveDown = i - j  
moveDown = 5 - 4  
moveDown = 1
```

Step 4: Add both parts

```
index = skipColumns + moveDown  
index = 12 + 1  
index = 13
```

So:

`M[5,4] -> A[13]`

23. Dry run 3: Find index of `M[5,3]`

We want:

`M[5,3]`

So:

```
n = 5  
i = 5  
j = 3
```


Step 1: Check condition

```
i >= j  
5 >= 3  
true
```

So the position is stored.

Step 2: Skip columns before column 3

Before column 3, we skip columns 1 and 2.

```
Column 1 has 5 values  
Column 2 has 4 values
```

So:

```
skipColumns = 5 + 4  
skipColumns = 9
```

Step 3: Move down inside column 3

Column 3 starts at:

```
M[3,3]
```

The target is:

```
M[5,3]
```

Movement:

```
M[3,3] -> M[4,3] -> M[5,3]
```

That is 2 steps.

Using the formula:

```
moveDown = i - j  
moveDown = 5 - 3  
moveDown = 2
```

Step 4: Add both parts

```
index = skipColumns + moveDown  
index = 9 + 2  
index = 11
```

So:

```
M[5,3] -> A[11]
```

24. Dry run 4: Use the full formula for M[5,3]

Now calculate M[5,3] using the full formula.

Formula:

```
index = n(j - 1) - ((j - 2)(j - 1) / 2) + (i - j)
```

Values:

```
n = 5  
i = 5  
j = 3
```

Substitute:

```
index = 5(3 - 1) - ((3 - 2)(3 - 1) / 2) + (5 - 3)
```

Simplify:

```
index = 5(2) - ((1)(2) / 2) + 2  
index = 10 - (2 / 2) + 2
```

```
index = 10 - 1 + 2  
index = 11
```

So:

```
M[5,3] -> A[11]
```

This matches the earlier dry run.

25. Dry run 5: Check `M[2,5]`

We want:

```
M[2,5]
```

So:

```
i = 2  
j = 5
```

Check the stored condition:

```
i >= j  
2 >= 5  
false
```

So `M[2,5]` is not stored.

Since:

```
i < j
```

this position is above the main diagonal.

Therefore:

```
M[2,5] = 0
```

It is a structural zero.

A `get` function should return:

0

A helper index function may return:

-1

26. Helper function for column-major index

A helper function can calculate the array index.

```
int getColumnMajorIndex(int n, int i, int j) {  
    if (i >= j) {  
        int skipColumns = n * (j - 1) - ((j - 2) * (j - 1) / 2);  
        int moveDown = i - j;  
        return skipColumns + moveDown;  
    }  
  
    return -1;  
}
```

This function returns:

The array index if the position is stored.
-1 if the position is not stored.

27. Why return `-1` in the helper function?

Valid array indexes start at 0.

Valid indexes are:

0, 1, 2, 3, 4, ...

So `-1` is not a valid array index.

That makes it useful as a signal.

It means:

This matrix position is not stored.

Example:

```
getColumnMajorIndex(5, 2, 5)
```

Since:

$$2 < 5$$

The function returns:

-1

28. Set operation idea

The `set` operation stores a value only if the position belongs to the lower triangle.

Rule:

If $i \geq j$, store the value.
If $i < j$, ignore the value.

Code idea:

```
void set(int A[], int n, int i, int j, int x) {  
    if (i >= j) {  
        int index = n * (j - 1) - ((j - 2) * (j - 1) / 2) + (i - j);  
        A[index] = x;  
    }  
}
```

This protects the lower triangular structure.

If someone tries to store a value above the diagonal, the function does nothing.

29. Set dry run: `set(A, 5, 5, 3, 99)`

Suppose the call means:

```
n = 5
i = 5
j = 3
x = 99
```

Check:

```
i >= j
5 >= 3
true
```

So the value can be stored.

Find the index:

```
index = 5(3 - 1) - ((3 - 2)(3 - 1) / 2) + (5 - 3)
index = 10 - 1 + 2
index = 11
```

Store:

```
A[11] = 99
```

So `99` is stored at the array position for `M[5,3]`.

30. Set dry run for a structural zero: `set(A, 5, 2, 5, 99)`

Suppose:

```
n = 5
i = 2
j = 5
x = 99
```

Check:

```
i >= j
2 >= 5
false
```

So the position is above the diagonal.

The function does nothing.

Why?

Because `M[2,5]` must remain zero in a lower triangular matrix.

31. Get operation idea

The `get` operation returns the value at a matrix position.

Rule:

```
If i >= j, calculate the index and return A[index].
If i < j, return 0.
```

Code idea:

```
int get(int A[], int n, int i, int j) {
    if (i >= j) {
        int index = n * (j - 1) - ((j - 2) * (j - 1) / 2) + (i - j);
        return A[index];
    }

    return 0;
}
```

Important difference:

Helper index function returns -1 for not stored.
get function returns 0 for not stored.

Why?

Because the actual matrix value at a structural zero position is 0.

32. Get dry run: `get(A, 5, 5, 3)`

Suppose:

```
n = 5  
i = 5  
j = 3
```

Check:

```
5 >= 3  
true
```

So the value is stored.

Calculate:

```
index = 5(3 - 1) - ((3 - 2)(3 - 1) / 2) + (5 - 3)  
index = 10 - 1 + 2  
index = 11
```

Return:

```
A[11]
```

33. Get dry run for a structural zero: `get(A, 5, 2, 5)`

Suppose:


```
n = 5
i = 2
j = 5
```

Check:

```
2 >= 5
false
```

So the position is not stored.

Return:

```
0
```

The function does not read the array.

It already knows the value must be zero because the position is above the main diagonal.

34. Display operation idea

The display operation prints the full matrix.

Even though we store only the lower triangle, the user should see the full `N x N` matrix.

Display uses nested loops:

```
for every row i
  for every column j
```

For each position:

```
If i >= j, print A[index].
If i < j, print 0.
```

Code idea:

```
void display(int A[], int n) {
    for (int i = 1; i <= n; i++) {
```

```

    for (int j = 1; j <= n; j++) {
        if (i >= j) {
            int index = n * (j - 1) - ((j - 2) * (j - 1) / 2) + (i - j);
            cout << A[index] << " ";
        } else {
            cout << "0 ";
        }
    }
    cout << endl;
}
}

```

Display still takes $O(N^2)$ time because it prints every visible matrix position.

35. Row-major vs column-major comparison table

Concept	Lower triangular row-major	Lower triangular column-major
Matrix type	Lower triangular	Lower triangular
Stored condition	$i \geq j$	$i \geq j$
Zero condition	$i < j$	$i < j$
Storage order	Row by row	Column by column
First stored group	Row 1	Column 1
Formula idea	Skip previous rows	Skip previous columns
Index formula	$i(i - 1)/2 + (j - 1)$	$n(j - 1) - ((j - 2)(j - 1)/2) + (i - j)$
Needs n in formula?	No	Yes

The most important point:

The condition does not change.
Only the storage order and index formula change.

36. Why column-major formula needs n

The column-major formula needs n because column sizes depend on the matrix dimension.

For `n = 5`:

```
Column 1 has 5 values  
Column 2 has 4 values  
Column 3 has 3 values  
Column 4 has 2 values  
Column 5 has 1 value
```

For `n = 4`:

```
Column 1 has 4 values  
Column 2 has 3 values  
Column 3 has 2 values  
Column 4 has 1 value
```

So the formula must know the size of the matrix.

That is why `n` appears in:

```
n(j - 1)
```

37. Why row-major formula does not need `n`

The row-major formula is:

```
index = i(i - 1)/2 + (j - 1)
```

It does not need `n` because rows before row `i` always store:

```
1 + 2 + 3 + ... + (i - 1)
```

This count depends only on `i`, not on the total matrix size.

But in column-major storage, column lengths depend on `n`, so the formula needs `n`.

38. Important connection to future lessons

Lesson 6 is also important because the same formula idea can help with upper triangular matrices later.

A lower triangular column-major formula is related to an upper triangular row-major formula.

The project later reuses triangular formulas by swapping the role of `i` and `j`.

This is part of the bigger pattern:

Change the matrix type or storage order.
Then change only the condition and index formula.
Keep the general program structure mostly the same.

39. Complexity of Lesson 6

Storage complexity

A lower triangular matrix stores:

$$N(N + 1) / 2$$

This is about half of a full matrix.

In Big-O notation, this is:

$$O(N^2)$$

Why is it still $O(N^2)$?

Because:

$$N(N + 1) / 2$$

still grows like N^2 as N becomes large.

So lower triangular storage is:

$O(N^2)$, but about half of full N^2 storage

40. Index calculation complexity

The formula:

$$\text{index} = n(j - 1) - ((j - 2)(j - 1) / 2) + (i - j)$$

uses only arithmetic.

It has:

No loop
No recursion
No search

So index calculation is:

$O(1)$

41. Set complexity

The `set` operation does:

One condition check
One formula calculation if needed
One array assignment if needed

So:

$\text{set}() = O(1)$

42. Get complexity

The `get` operation does:

```
One condition check
One formula calculation if needed
One array lookup if needed
```

So:

```
get() = O(1)
```

43. Display complexity

The `display` operation prints the full visible matrix.

For an `N x N` matrix, there are:

```
N² visible positions
```

So:

```
display() = O(N²)
```

Even though the matrix stores only the lower triangle, the display still prints every row and every column.

44. Complexity table

Operation	Time complexity	Space complexity	Reason
Storage	—	<code>O(N²)</code> , about half of full storage	Stores <code>N(N + 1)/2</code> values.
Index calculation	<code>O(1)</code>	—	Uses direct arithmetic.
<code>set()</code>	<code>O(1)</code>	—	One condition and one formula.

Operation	Time complexity	Space complexity	Reason
get()	$O(1)$	—	One condition and one possible array lookup.
display()	$O(N^2)$	—	Prints every visible matrix position.

45. Common beginner mistakes

Mistake 1: Using the row-major formula in Lesson 6

Wrong for column-major:

```
index = i * (i - 1) / 2 + (j - 1);
```

Correct for column-major:

```
index = n * (j - 1) - ((j - 2) * (j - 1) / 2) + (i - j);
```

Reason:

Row-major skips rows.
Column-major skips columns.

Mistake 2: Forgetting to check $i \geq j$

Wrong idea:

Always calculate the formula.

Correct idea:

Check $i \geq j$ first.

If $i < j$, the position is not stored.

Mistake 3: Thinking column-major changes the matrix shape

Wrong idea:

```
Column-major means a different matrix.
```

Correct idea:

```
Column-major means a different storage order.
```

The lower triangular shape is the same.

Only the 1D array order changes.

Mistake 4: Forgetting `n` in the formula

Wrong:

```
index = (j - 1) - ((j - 2) * (j - 1) / 2) + (i - j);
```

Correct:

```
index = n * (j - 1) - ((j - 2) * (j - 1) / 2) + (i - j);
```

Reason:

Column sizes depend on the matrix size `n`.

Mistake 5: Returning `-1` from `get()`

Wrong for `get`:

```
return -1;
```

Correct for `get`:

```
return 0;
```


Reason:

If the position is above the diagonal, the actual matrix value is zero.

`-1` is useful for helper index functions, not for the final matrix value.

Mistake 6: Thinking storage is `O(N)`

Wrong idea:

Lower triangular storage is `O(N)`.

Correct idea:

Lower triangular storage is `O(N2)`, but it uses about half of full matrix storage.

Reason:

It stores:

$N(N + 1) / 2$

This still grows like `N2`.

46. Practice questions

Question 1

Is `M[4, 2]` stored?

Check:

`4 >= 2`

Answer:

Yes, it is stored.

Question 2

Is $M[2,4]$ stored?

Check:

$2 \geq 4$

This is false.

Answer:

No, it is a structural zero.

Question 3

For $n = 5$, find the index of $M[4,4]$.

Answer:

$M[4,4] \rightarrow A[12]$

Question 4

For $n = 5$, find the index of $M[5,4]$.

Answer:

$M[5,4] \rightarrow A[13]$

Question 5

For $n = 5$, find the index of $M[5,3]$.

Answer:

M[5,3] -> A[11]

Question 6

What does `getColumnMajorIndex(5, 2, 5)` return?

Check:

2 >= 5

This is false.

Answer:

-1

Reason:

`M[2,5]` is above the diagonal, so it is not stored.

Question 7

What should `get(A, 5, 2, 5)` return?

Answer:

0

Reason:

`M[2,5]` is a structural zero.

47. Key formulas from Lesson 6

Stored condition

$$i \geq j$$

Zero condition

$$i < j$$

Stored values

$$N(N + 1) / 2$$

Structural zeros

$$N(N - 1) / 2$$

Column-major index formula

$$\text{index} = n(j - 1) - ((j - 2)(j - 1) / 2) + (i - j)$$

Formula meaning

$$\text{index} = \text{skipColumns} + \text{moveDown}$$

Skip previous columns

$$\text{skipColumns} = n(j - 1) - ((j - 2)(j - 1) / 2)$$

Move inside current column

$$\text{moveDown} = i - j$$

48. Full helper code summary

```
int getColumnMajorIndex(int n, int i, int j) {
    if (i >= j) {
        int skipColumns = n * (j - 1) - ((j - 2) * (j - 1) / 2);
        int moveDown = i - j;
        return skipColumns + moveDown;
    }

    return -1;
}
```

49. Full set code summary

```
void set(int A[], int n, int i, int j, int x) {
    if (i >= j) {
        int index = n * (j - 1) - ((j - 2) * (j - 1) / 2) + (i - j);
        A[index] = x;
    }
}
```

50. Full get code summary

```
int get(int A[], int n, int i, int j) {
    if (i >= j) {
        int index = n * (j - 1) - ((j - 2) * (j - 1) / 2) + (i - j);
        return A[index];
    }

    return 0;
}
```

51. Full display code summary

```
void display(int A[], int n) {
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
```

```

        if (i >= j) {
            int index = n * (j - 1) - ((j - 2) * (j - 1) / 2) + (i - j);
            cout << A[index] << " ";
        } else {
            cout << "0 ";
        }
    }
    cout << endl;
}
}

```

52. Beginner mental model

Think of the full matrix as what the user sees.

Example:

1	0	0	0	0
2	3	0	0	0
4	5	6	0	0
7	8	9	10	0
11	12	13	14	15

Think of the 1D array as what the computer stores.

For column-major storage, the computer stores column by column:

```
A = [1, 2, 4, 7, 11, 3, 5, 8, 12, 6, 9, 13, 10, 14, 15]
```

The zeros above the diagonal are not stored.

When the user asks for a stored value, the program calculates the index and reads from the array.

When the user asks for a structural zero, the program returns `0` immediately.

53. The most important thinking pattern

For lower triangular column-major mapping, always ask:

Question 1

Is the position stored?

```
i >= j
```

If no, return zero or `-1` depending on the function.

Question 2

How many stored values come before this column?

```
skipColumns = n(j - 1) - ((j - 2)(j - 1) / 2)
```

Question 3

How far do I move down inside this column?

```
moveDown = i - j
```

Final step

Add them:

```
index = skipColumns + moveDown
```

54. Exit questions

You should be able to answer these before moving to Lesson 7:

1. What is a lower triangular matrix?
2. What condition tells us a position is stored?
3. What condition tells us a position is a structural zero?
4. What does column-major mean?
5. How is column-major different from row-major?
6. How many values are stored in an `N x N` lower triangular matrix?
7. Why does column 1 have `N` stored values?
8. Why does column `N` have only 1 stored value?
9. What is the column-major index formula?
10. What does `skipColumns` mean?

11. What does `moveDown` mean?
 12. Why does the formula use `n`?
 13. What index does `M[4,4]` map to when `n = 5`?
 14. What index does `M[5,4]` map to when `n = 5`?
 15. What index does `M[5,3]` map to when `n = 5`?
 16. Why does `M[2,5]` return zero?
 17. Why can a helper function return `-1`?
 18. What is the storage complexity?
 19. What is the index calculation complexity?
 20. What is the main lesson from Lesson 6?
-

55. Short answers to exit questions

1. A lower triangular matrix stores values on and below the main diagonal.
 2. A position is stored when `i >= j`.
 3. A position is a structural zero when `i < j`.
 4. Column-major means storing values column by column.
 5. Row-major stores row by row; column-major stores column by column.
 6. It stores $N(N + 1) / 2$ values.
 7. Column 1 has `N` stored values because it starts at row 1 and continues down to row `N`.
 8. Column `N` has only one stored value because it starts at `M[N,N]`.
 9. The formula is `n(j - 1) - ((j - 2)(j - 1)/2) + (i - j)`.
 10. `skipColumns` counts stored values before column `j`.
 11. `moveDown` tells how far to move downward inside column `j`.
 12. The formula uses `n` because column sizes depend on the matrix dimension.
 13. `M[4,4] -> A[12]`.
 14. `M[5,4] -> A[13]`.
 15. `M[5,3] -> A[11]`.
 16. `M[2,5]` returns zero because `2 < 5`, so it is above the diagonal.
 17. A helper function can return `-1` because `-1` is not a valid array index, so it signals "not stored."
 18. Storage complexity is $O(N^2)$, but about half of full matrix storage.
 19. Index calculation complexity is $O(1)$.
 20. The main lesson is: for lower triangular column-major storage, keep the condition `i >= j`, but change the index formula to skip columns instead of rows.
-

56. Lesson 6 final summary

Lesson 6 teaches how to store a lower triangular matrix using column-major order.

A lower triangular matrix stores values where:

$$i \geq j$$

It treats positions where:

$$i < j$$

as structural zeros.

The storage size is:

$$N(N + 1) / 2$$

This is about half of a full matrix.

Column-major storage means storing values column by column.

For a `5 x 5` lower triangular matrix:

```
Column 1 has 5 values
Column 2 has 4 values
Column 3 has 3 values
Column 4 has 2 values
Column 5 has 1 value
```

The column-major index formula is:

$$\text{index} = n(j - 1) - ((j - 2)(j - 1) / 2) + (i - j)$$

The easiest way to remember the formula is:

$$\text{index} = \text{skip previous columns} + \text{move down inside current column}$$

Important examples for `n = 5`:

```
M[4,4] -> A[12]
M[5,4] -> A[13]
M[5,3] -> A[11]
M[2,5] -> structural zero, so get returns 0
```

The main difference between Lesson 5 and Lesson 6 is:

Lesson 5: row-major, skip previous rows.
Lesson 6: column-major, skip previous columns.

The matrix condition stays the same.

Only the storage order and index formula change.